

PRÁCTICA 4

Acceso físico con NFC

Voz · Estrategia

Universidad La Salle México

Aplicaciones Móviles · 900CIB

Equipo GIGN

Aristide Cordonnier · Mathis Gil · Oscar Buat

Docente: M. en C. Felipe Mandujano Gómez

Objetivo y marco teórico

Objetivo

Crear una app con React Native y Expo para usar el teléfono como credencial. La app debe permitir iniciar sesión, mostrar los datos del usuario y enviar su número al lector NFC. El proyecto es del equipo GIGN, para un callcenter de estrategias políticas.

React Native y Expo

React Native permite crear pantallas para el teléfono con JavaScript. Expo ayuda a ejecutar y compilar la app. Usamos Expo Router para pasar entre las pantallas de acceso, credencial y créditos.

Inicio de sesión

Una API recibe una petición de la app y devuelve una respuesta. En esta práctica usamos la API MonsterBallGo. El formulario envía el usuario y la contraseña convertida con SHA-256, como pide el ejemplo del profesor. SHA-256 produce una cadena de texto a partir de la contraseña. La petición viaja por HTTPS. La app no guarda la contraseña (Mandujano Gómez, s. f.-b).

NFC y APDU

NFC permite comunicar dos dispositivos al acercarlos. Android HCE hace que el teléfono responda como una tarjeta. El lector envía un comando APDU y la app devuelve el número de usuario en cuatro bytes. El AID es el código que usa el lector para encontrar nuestro servicio: F012345678 (Google, s. f.).

En este proyecto, el número se envía en orden little endian: primero el byte de menor peso. Ese orden coincide con la copia a uint32_t que hace ScannerHTTP en el ESP32. Al final se agregan 90 00, que indican que el comando se respondió correctamente. El lector consulta al servidor para decidir el acceso.

El montaje usa un Arduino Nano ESP32 y un lector PN532. En el proyecto del profesor se conectan por HSU, un modo de comunicación serial.

Material y equipo

Recurso	Uso
Computadora con Node.js y JDK	Editar el código y compilar la app
React Native 0.88 y Expo 58	Crear las pantallas
Teléfono Android con NFC y HCE	Instalar la app y probar el lector
Arduino Nano ESP32 y PN532	Leer el número del teléfono
Tres cuentas MonsterBallGo	Probar a cada integrante de GIGN
Servidor y panel del profesor	Consultar y mostrar el resultado del acceso

La demostración usa un teléfono Android compatible con NFC y el lector proporcionado por el profesor.

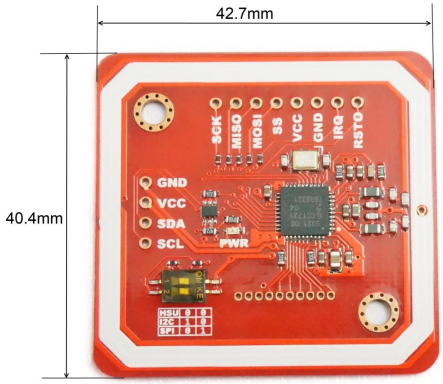


Figura 1. PN532 v3. Imagen del manual de Elechouse, página 2.

El nombre de la app es Voz · Estrategia. En los créditos aparecen los logos de React, Expo, Android, Kotlin, Arduino y Node.js. Los recursos están guardados dentro del proyecto.

Cuentas del equipo

Integrante	Usuario	Número de usuario
Aristide Cordonnier	aristidecordonnier	767108734
Mathis Gil	mathisgil	651791183
Oscar Buat	oscarbuat	877063652

Desarrollo experimental

Organización del proyecto

La app nueva está en expo-client. Se conservaron los directorios app, server y firmware del profesor. Así seguimos usando el mismo lector y servidor.

App.js guarda la sesión. Los tres archivos de src/app muestran el acceso, la credencial y los créditos. src/api.js hace la petición de login. src/nfc.js conecta la pantalla con el servicio NFC escrito en Kotlin.

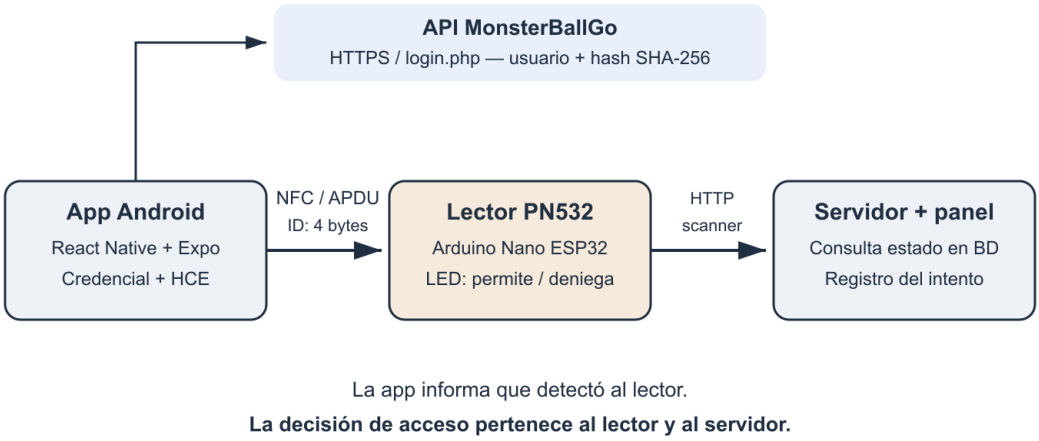


Figura 2. La app inicia sesión con la API. El Arduino consulta al servidor para decidir el acceso.

Login y credencial

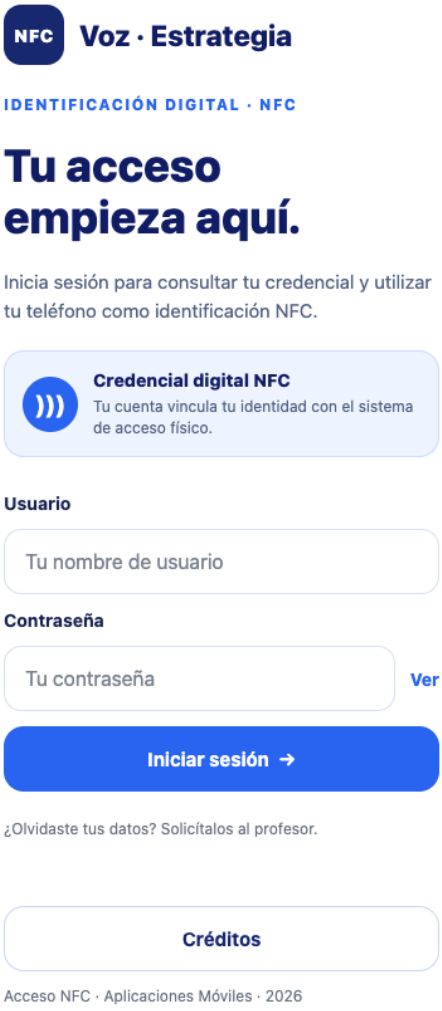
El usuario escribe su cuenta y contraseña. La app comprueba que los dos campos tengan datos y envía el formulario a https://monsterballgo.com/api/login.php. Usa los campos user, pass y token del ejemplo del profesor. Si hay un error, lo muestra en la misma pantalla. La espera máxima es de 15 segundos.

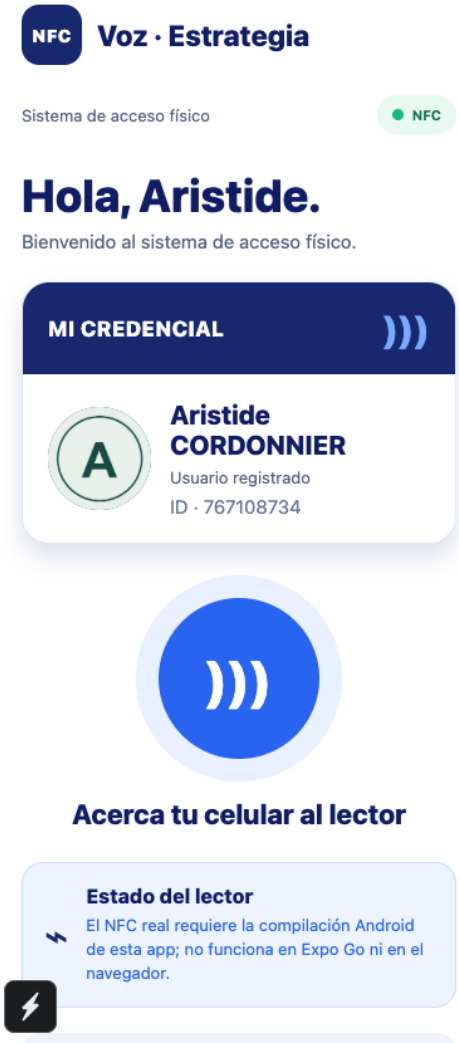
Si la API devuelve un ID válido, la app abre la credencial. Muestra la foto, el nombre completo y el número de usuario. Si la foto no carga, aparecen las iniciales. El botón Créditos abre los nombres del equipo, la fecha y los logos de las tecnologías.

Repositorio del equipo: código, APK e informe.
https://github.com/aristocratte/NFC

Pantallas de la app

Se inició sesión desde el navegador con las tres cuentas reales del equipo y la API del profesor. Estas capturas muestran el formulario de acceso y la credencial de Aristide después de una conexión correcta. Los resultados y las imágenes no incluyen contraseñas.





Figuras 3 y 4. Formulario de acceso y credencial de Aristide tras un login real con la API.

La interfaz se puede revisar en el navegador. Para usar NFC hay que instalar el APK en Android, porque Expo Go no incluye nuestro servicio HCE.

Pruebas con los tres usuarios

Se consultó el perfil público de cada integrante en MonsterBallGo y se descargó su imagen por HTTPS. Las seis peticiones respondieron HTTP 200. Los nombres, números e imágenes coinciden con el registro de la tarea de fotos de perfil.



Figura 5. Perfiles públicos reales de Aristide, Mathis y Oscar. Se recortaron los datos de contacto.

Credenciales revisadas

La API aceptó los datos de acceso de Aristide, Mathis y Oscar: tres respuestas HTTP 200 con el ID esperado. Para cada cuenta se comprobaron el nombre completo, el número, la foto y el cierre de sesión. Después de salir, abrir la ruta de la credencial devolvió al formulario de acceso.



Figura 6. Las tres credenciales obtenidas con conexiones reales a MonsterBallGo desde la app en el navegador.

No hubo que cambiar otra vez las fotos. Las URLs de las tres imágenes siguen siendo las guardadas en el trabajo anterior. También se comprobó que la petición enviaba la contraseña convertida con SHA-256 y que ni la contraseña ni ese valor quedaban en localStorage o sessionStorage. Al salir, el campo de contraseña quedó vacío.

Pruebas del servicio NFC

El servicio está en modules/acceso-nfc y tiene dos clases Kotlin. Una recibe el ID desde JavaScript. La otra responde al lector. Al cerrar sesión se borra el ID y se desactiva la respuesta. Si el proceso de la app se reinicia, hay que iniciar sesión otra vez.

Orden de los cuatro bytes

Al comparar la app con ScannerHTTP se encontró que el envío inicial estaba en big endian. El firmware copia esos bytes directamente a un uint32_t del ESP32, que usa little endian. Se corrigió el orden en la app y se volvió a compilar el APK. El firmware del profesor se conservó.

Usuario	ID	Respuesta completa
aristidecordonnier	767108734	7E 26 B9 2D 90 00
mathisgil	651791183	4F 8B D9 26 90 00
oscarbuat	877063652	E4 ED 46 34 90 00

Los primeros cuatro bytes contienen el número. Los dos últimos, 90 00, indican una respuesta APDU correcta. No significan que el usuario tenga acceso.

Ejecución sin teléfono

Se ejecutó en la computadora la clase Kotlin compilada que responde al lector. Se sustituyeron las funciones de Android, como la vibración y el almacenamiento. Pasaron 20 comprobaciones: los tres IDs, los límites 1 y 4294967295, SELECT con o sin el byte final 00, comandos inválidos y ausencia de sesión.

Caso	Respuesta comprobada
SELECT correcto con sesión	Cuatro bytes del ID y 90 00
Sesión cerrada o ID ausente	69 85; no entrega el ID
AID o longitud incorrectos	6A 82; no dispara la detección
Comando nulo	6F 00

Estos resultados verifican el cálculo y las respuestas del código. No miden la comunicación por radio, la vibración física ni el funcionamiento de HCE en un teléfono. La app sigue sin llamar al endpoint del escáner y sin mostrar una decisión de acceso.

Prueba con el montaje

Instalar el APK, activar NFC e iniciar sesión. Acercar el teléfono desbloqueado al PN532, revisar la detección en la app y comparar el ID del panel con la credencial. Repetir con cada integrante, incluido el que bloquee el profesor, y fotografiar el LED del mismo intento.

Pruebas reales del servidor y del panel

Desde la computadora se consultó el endpoint HTTPS del escáner con los tres IDs del equipo. Cada consulta respondió HTTP 200 y si. El panel real recibió los datos por WebSocket, mostró el nombre y el número correctos y pintó la tarjeta en verde. No se usó NFC para producir estos eventos ni se cambió el estado de las cuentas.

Usuario	Respuesta API	Panel real
aristidecordonnier	HTTP 200 · si	Nombre e ID correctos; verde
mathisgil	HTTP 200 · si	Nombre e ID correctos; verde
oscarbuat	HTTP 200 · si	Nombre e ID correctos; verde



Figura 7. Panel real del profesor después de consultas directas a la API desde la computadora. No son escaneos NFC. Se recortaron los datos de contacto.

Cuenta bloqueada y otros errores

Los tres IDs recibieron autorización en estas consultas. El caso bloqueado se probó por separado con datos simulados: el código scanner.js respondió si para active y no para banned con cada ID. También negó un ID desconocido, un token incorrecto y una petición sin ID. Pasaron nueve casos del servidor local y seis del HTML del panel con mensajes simulados.

Estas pruebas muestran que la base real reconoce los tres IDs y que el panel recibe los eventos. Para la demostración exigida falta que el teléfono entregue el ID por NFC y que el Arduino encienda el LED correspondiente. El profesor debe bloquear una cuenta para comprobar también ese rechazo en el montaje real.

Análisis de resultados y mejoras

Prueba	Resultado observado
Perfiles e imágenes de los tres usuarios	Seis respuestas HTTP 200; datos e imágenes coinciden
Login real con las tres cuentas	HTTP 200; foto, nombre e ID correctos; cierre de sesión y ruta protegida correctos
Cuenta ficticia inexistente	API real devuelve Failed to login; la app muestra el error
API del escáner y panel reales	Los tres IDs reciben si; sus nombres e IDs aparecen en verde en el panel
Interfaz y errores simulados	20 comprobaciones correctas con el formato de nombres de la API real
Servicio APDU en computadora	20 comprobaciones correctas con funciones Android sustituidas
Servidor y panel con datos simulados	9 casos del servidor y 6 del panel correctos
Compilación Android	APK actualizado generado; firma verificada

Qué muestran las pruebas

El login real funcionó con las tres cuentas. La primera conexión mostró una diferencia que las respuestas simuladas no habían revelado: la API usa firstname y lastname. La pantalla esperaba first_name y last_name y mostraba el usuario en lugar del nombre completo. Se adaptó src/api.js y se repitieron las conexiones. Las tres credenciales mostraron los datos correctos y el cierre de sesión protegió su pantalla.

El error de orden de bytes podía cambiar el usuario recibido por el lector. Por ejemplo, los bytes iniciales del ID de Aristide se habrían interpretado como otro número. Después del cambio, los tres IDs se recuperaron sin alteración al leerlos como lo hace el firmware. Esta fue una corrección de compatibilidad, no de diseño visual.

Se comprobó el recorrido formulario, API, credencial y cierre de sesión en el navegador, además de las consultas reales al servidor del escáner y la recepción del panel. Falta repetir el recorrido con el APK en Android y entregar el ID por NFC al PN532. No se contó con ese teléfono: la comunicación por radio, la vibración y los LED del montaje no se han verificado.

Mejoras a futuro

El ID es fijo y aparece en el perfil público. Conocerlo no demuestra quién tiene el teléfono. Para un uso en empresa proponemos una respuesta distinta en cada lectura, comprobada por el servidor, y HTTPS entre el Arduino y el servidor. El panel podría filtrar por usuario y distinguir un bloqueo de un fallo de red. También hace falta una tarjeta de préstamo para quien no pueda usar el móvil.

Conclusiones y bibliografía

Conclusiones

A Voz · Estrategia le proponemos este sistema para organizar la entrada a su callcenter de estrategias políticas. En una campaña puede entrar personal temporal y cambiar la distribución de turnos. Una cuenta por trabajador permite retirar su acceso cuando acaba el contrato, sin esperar a recuperar una tarjeta. En recepción, el panel ayuda a comprobar qué cuenta se presentó cuando hay una incidencia. Es útil en un lugar donde se manejan listas de contactos y guiones de llamadas. Antes de comprar un despliegue completo, la empresa debería pedir una prueba con su lector, sus turnos y una cuenta bloqueada. También habría que proteger mejor el ID: identificar una cuenta con un número público no basta para proteger esos datos.

Desde el punto de vista del trabajador, usar el teléfono como credencial puede ser cómodo, pero no debería ser la única forma de entrar. Una batería descargada, un celular averiado o un modelo sin NFC pueden dejarlo fuera aunque tenga permiso para trabajar. También se le obliga a usar un equipo personal para una necesidad de la empresa. Por eso proponemos mantener una tarjeta de préstamo o un acceso por recepción. El empleado debe saber qué datos se guardan y quién puede ver sus entradas. Además, una lectura de la credencial no demuestra toda su jornada ni debería usarse por sí sola para decidir si trabajó o no.

Bibliografía

Espressif Systems. (s. f.). ESP32-S3 Technical Reference Manual, capítulo System and Memory.
https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf

Elechouse. (s. f.). PN532 NFC RFID Module User Guide, Version 3.
https://www.elechouse.com/elechouse/images/product/PN532_module_V3/PN532_%20Manual_V3.pdf

Expo. (s. f.-a). Expo SDK 58 reference.
https://docs.expo.dev/versions/v58.0.0/

Expo. (s. f.-b). Add a new module to an existing application.
https://docs.expo.dev/modules/get-started/

Google. (s. f.). Host-based card emulation overview. Android Developers.
https://developer.android.com/develop/connectivity/nfc/hce

Mandujano Gómez, F. (s. f.-a). NFCAccesoFisico [Código fuente]. GitHub.
https://github.com/fmg2099/NFCAccesoFisico

Mandujano Gómez, F. (s. f.-b). expo-login-registro [Código fuente]. GitHub.
https://github.com/fmg2099/expo-login-registro/blob/master/app/index.tsx

Meta. (s. f.). Core Components and APIs. React Native.
https://reactnative.dev/docs/components-and-apis

Devicon contributors. (s. f.). Devicon [Logos]. GitHub.
https://github.com/devicons/devicon